

Analisi computazionale di architetture JEPA-like su acceleratori per sistemi embedded ed Edge AI

Relazione tecnica per il corso di IoT e Smart Systems LM-32

Nunzio Saitta

8 luglio 2026

Indice

1	Introduzione	2
1.1	L'evoluzione dei vision model	2
1.2	Il Self-Supervised Learning (SSL)	2
1.3	Joint Embedding Predictive Architecture (JEPA)	2
1.4	DINOv2	2
2	Architettura del modello	3
2.1	Pipeline di pre-processing e tokenizzazione	3
2.2	Struttura del transformer	3
3	Metodologia	5
4	Risultati sperimentali	6
4.1	Throughput e latenza	6
4.2	Analisi dei sottomoduli	6
5	Deployment embedded ed Edge AI	8
5.1	Criticità del cloud	8
5.2	Edge AI	8
6	Conclusioni	9

1 Introduzione

1.1 L'evoluzione dei vision model

La Computer Vision è stata a lungo dominata dalle reti neurali convoluzionali (CNN), che sfruttano proprietà intrinseche delle immagini e applicano l'operazione di convoluzione. Tuttavia, quest'ultime mostrano limiti nella cattura di relazioni e dipendenze a lunghissimo raggio all'interno di un'immagine.

L'introduzione dei Vision transformer (ViT), nati in seguito all'enorme successo avuto dai transformer nel campo dell'elaborazione del linguaggio naturale (NLP), ha rivoluzionato questo paradigma. I ViT eliminano i layer convoluzionali lungo la profondità della rete, trattando l'immagine come una sequenza di tasselli lineari (**patch**). Il cuore pulsante dei ViT è il meccanismo di *Self-Attention*, che permette a ogni patch di interagire con tutte le altre simultaneamente, catturando relazioni globali sin dai primi livelli della rete.

Questo cambio di paradigma comporta un costo computazionale non indifferente, dato che le CNN operano con complessità linearmente dipendente dal numero di pixel mentre il meccanismo di attenzione globale dei transformer introduce una complessità spaziale e temporale quadratica rispetto al numero di token ($O(N^2)$), ponendo severe sfide per il deployment su nodi IoT e piattaforme di Edge AI.

1.2 Il Self-Supervised Learning (SSL)

Il successo dei modelli Deep Learning tradizionali è storicamente legato alla disponibilità di enormi dataset annotati manualmente (es. ImageNet). Nel contesto IoT e dei sistemi industriali smart, l'annotazione manuale dei dati è spesso costosa o soggetta a forti bias. Il *SSL* consente ai modelli di apprendere rappresentazioni visive robuste e generalizzabili direttamente da dati non etichettati.

Attraverso compiti pretestuosi, come il mascheramento di porzioni di immagine o la massimizzazione della similarità tra viste aumentate della stessa immagine (contrastive learning), il modello sviluppa una comprensione semantica profonda. Le feature estratte possono poi essere sfruttate nei task a valle come la classificazione o il tracciamento di oggetti con un addestramento aggiuntivo meno oneroso o addirittura minimo.

1.3 Joint Embedding Predictive Architecture (JEPA)

Joint Embedding Predictive Architecture (JEPA) ridefinisce le fondamenta dell'apprendimento auto-supervisionato, superando le inefficienze dei modelli generativi classici:

- i modelli basati sulla pixel reconstruction (come i Masked Autoencoders) tentano di riprodurre fedelmente ogni pixel mancante di un'immagine. Questo approccio costringe la rete a spendere ingenti risorse computazionali ed energetiche per modellare dettagli che possono rilevarsi irrilevanti dal punto di vista semantico, come il rumore di fondo di un sensore d'immagine.
- si elimina la necessità di generare pixel nello spazio di input. L'architettura prevede la scomposizione dell'immagine in blocchi, mappando le diverse porzioni in uno spazio latente (**embedding**). Il task predittivo non consiste nel ricreare i pixel mancanti, bensì nel predire l'embedding target partendo dall'embedding di contesto.

1.4 DINOv2

Il modello **DINOv2** sviluppato da Meta AI rappresenta un'espressione ingegneristica del paradigma JEPA combinato con l'architettura vision transformer. DINOv2 unisce un approccio di apprendimento discriminativo a livello di immagine (tramite una perdita contrastiva multi-crop) con un approccio predittivo a livello di patch (simile al masked image modeling, ma calcolato interamente nello spazio latente degli embedding). Grazie a questo rigido protocollo di ottimizzazione e a un dataset di addestramento curato e massivo, DINOv2 estrae feature visive che superano i modelli supervisionati tradizionali in quasi tutti i benchmark di Computer Vision, senza mai aver visto un'etichetta durante il pre-addestramento.

Nel contesto di questa analisi, viene preso in esame il sottomodulo `dinov2_vits14` (Vision transformer Small, con patch size di 14×14 pixel). Questa configurazione rappresenta un ottimo candidato per applicazioni IoT e Smart Systems, bilanciando un'eccellente capacità rappresentativa con un numero di parametri compatibili con acceleratori hardware di classe Edge.

2 Architettura del modello

L'analisi dell'architettura di `dinov2.vits14` richiede una comprensione approfondita di come i modelli basati ViT abbiano modificato il paradigma dell'elaborazione visiva rispetto alle reti convoluzionali tradizionali. La versione *Small* di DINOv2, caratterizzata da circa 22 milioni di parametri operanti su blocchi di patch da 14 pixel, rappresenta un caso di studio ottimale per i sistemi IoT e Smart Systems. Questo modello, infatti, pur ereditando la complessità computazionale tipica dei transformer basati su meccanismi di self-attention, mantiene un'occupazione di memoria e un volume di parametri sufficientemente contenuti da consentirne il deployment su nodi periferici. Il principio fondamentale risiede nella sua capacità di convertire una matrice bidimensionale densa, quale è un frame d'immagine grezzo, in una sequenza di vettori latenti ad alto contenuto informativo. In un contesto IoT, dove i dati provengono da sensori visivi eterogenei o telecamere di sorveglianza dislocate sul territorio, questa trasformazione deve avvenire idealmente rispettando vincoli stringenti di latenza, determinismo temporale e consumo energetico.

2.1 Pipeline di pre-processing e tokenizzazione

Prima che l'informazione possa essere digerita dai blocchi di calcolo del transformer, è necessaria una fase di pre-elaborazione strutturata che uniformi l'input. Il frame visivo viene inizialmente sottoposto a operazioni di ridimensionamento e normalizzazione, venendo ricondotto alla risoluzione standard di 224×224 pixel con i classici tre canali cromatici RGB. Questo passaggio permette di stabilire una dimensione d'ingresso fissa così da determinare a priori il carico computazionale di ogni successiva inferenza, rendendo l'allocazione delle risorse di memoria RAM e i tempi di esecuzione stabili e prevedibili.

Una volta normalizzata l'immagine, il modello esegue l'operazione fondamentale di tokenizzazione, nota come *patch extraction*. L'immagine di 224×224 pixel viene idealmente suddivisa in una griglia di tessere disgiunte, denominate patch, ciascuna avente una dimensione geometrica di 14×14 pixel. Da questa suddivisione spaziale scaturisce una griglia composta da esattamente 16 patch per lato, determinando un totale di 256 patch geometriche complessive.

Ogni singola patch rappresenta una porzione tridimensionale dell'immagine originale, quantificabile in $14 \times 14 \times 3 = 588$ valori numerici di pixel. Questa informazione visiva grezza viene successivamente convogliata all'interno di uno strato di proiezione lineare (*Linear Projection*), implementato tramite una convoluzione bidimensionale con passo pari alla dimensione della patch stessa. L'obiettivo di questo layer è proiettare i 588 valori di partenza in uno spazio di embedding compatto a dimensionalità fissa, definita dalla costante architetturale $D = 384$ per la configurazione *Small*.

Per completare la sequenza di input e prepararla per i successivi stadi di calcolo, vengono introdotte due componenti fondamentali per l'apprendimento auto-supervisionato e la conservazione dell'informazione spaziale. All'inizio della sequenza dei 256 vettori di embedding viene inserito un parametro pre-appreso denominato [CLS] **token** (Classification Token), portando la lunghezza totale del tensore a 257 elementi. Questo elemento ha il compito di raccogliere e sintetizzare l'informazione globale proveniente da tutte le patch durante il passaggio nei vari blocchi della rete.

Subito dopo, l'intera sequenza viene addizionata elemento per elemento con un tensore di *Positional Embedding* unidimensionale apprendibile. Poiché i meccanismi di attenzione pura trattano i token come un insieme non ordinato, l'aggiunta di questi vettori di posizione è l'unico modo per preservare la struttura geometrica e le relazioni spaziali originarie del frame visivo. Nei sistemi di monitoraggio intelligente, la conoscenza della posizione relativa degli elementi all'interno della scena è un requisito cruciale per garantire la correttezza semantica delle feature estratte.

2.2 Struttura del transformer

Il nucleo computazionale ed energetico di `dinov2.vits14` è rappresentato da una pila di 12 blocchi transformer identici, disposti sequenzialmente in cascata. La caratteristica portante di questa struttura risiede nel fatto che la dimensione interna dei vettori latenti rimane rigorosamente costante a 384 canali lungo l'intera esecuzione.

Ciascuno dei 12 blocchi opera applicando internamente due macro-moduli principali, preceduti da strati di normalizzazione (*Layer Normalization*) e stabilizzati sia da funzioni di scalatura dei vettori (*LayerScale*) sia da connessioni residue. La presenza delle connessioni residue garantisce che l'informazione possa fluire intatta attraverso i blocchi, riducendo il rischio di degradazione dei gradienti e permettendo a ciascun livello di specializzarsi su dettagli a diversa granularità semantica.

Il primo sottomodulo critico incontrato dal tensore prende il nome di **attn**, responsabile del calcolo della *Multi-Head Self-Attention*. In questa specifica architettura, l'attenzione viene ripartita su 6 teste indipendenti che operano in parallelo. Ciascuna testa permette a ogni singolo token dei 257 presenti nella sequenza di confrontarsi direttamente con tutti gli altri, valutando le reciproche correlazioni e costruendo una mappa di dipendenze globali indipendente dalla distanza fisica dei pixel sul frame originario.

Dal punto di vista dell'analisi empirica e delle performance sistemiche, il modulo **attn** costituisce il principale collo di bottiglia computazionale quando il modello viene eseguito su processori tradizionali privi di parallelismo massivo, come le CPU standard dei nodi IoT industriali. L'operazione di calcolo delle matrici di attenzione presenta infatti una complessità intrinseca di tipo quadratico rispetto al numero di token, traducendosi in pesanti operazioni di moltiplicazione di matrici che saturano rapidamente le unità di calcolo sequenziali.

Il secondo sottomodulo, denominato **mlp** (*Multi-Layer Perceptron*), riceve l'output stabilizzato del livello di attenzione ed è costituito da una rete densa *feed-forward*. Questo modulo opera in modo indipendente su ciascun token della sequenza, applicando una proiezione lineare che espande temporaneamente la dimensione interna di un fattore pari a quattro, raggiungendo una dimensionalità intermedia di 1536 canali. Dopo l'applicazione di una funzione di attivazione non lineare, il tensore viene nuovamente compresso e riportato alla dimensione originaria di 384 canali.

La natura strutturale del modulo **mlp** lo rende profondamente diverso dal blocco di attenzione in termini di profilo di carico hardware. Trattandosi di proiezioni dense applicabili in parallelo su ciascun token, questo modulo trae un beneficio computazionale immenso dall'architettura multi-core delle GPU e degli acceleratori hardware.

3 Metodologia

Per comprendere l'impatto reale di un modello ViT, l'analisi comparativa è stata interamente sviluppata nell'ambiente cloud Google Colab. Questa scelta ha permesso di alternare facilmente l'esecuzione del codice tra un processore tradizionale (CPU) e un acceleratore grafico (GPU).

Uno dei requisiti fondamentali del progetto era misurare i tempi di esecuzione interni al modello, isolando il comportamento dei singoli layer (Self-Attention e MLP). Per raggiungere questo obiettivo, sono stati utilizzati i *Forward Hooks*.

Un hook può essere immaginato come una funzione di callback che viene agganciata a uno specifico blocco del modello. Nell'esperimento, la sonda è stata inserita all'interno di ognuno dei 12 blocchi del transformer, distinguendo nettamente la componente legata all'attenzione visiva (`attn`) da quella densa e lineare (`mlp`). Per ogni sottomodulo sono stati combinati due strumenti:

- Il Pre-Hook (`register_forward_pre_hook`), che si attiva un istante prima che il blocco inizi a elaborare i dati in ingresso, catturando il timestamp di avvio.
- L'Hook classico (`register_forward_hook`), che interviene immediatamente dopo la fine del calcolo, registrando il timestamp di conclusione e calcolando la latenza netta per differenza.

Durante la configurazione dei test su GPU è emerso un aspetto cruciale legato alla natura dei moderni hardware paralleli. Di default, le operazioni invocate sui dispositivi CUDA avvengono in modalità asincrona dunque quando il processore invia un comando alla GPU (come una pesante moltiplicazione di matrici), non attende che l'operazione sia conclusa, ma riprende immediatamente il controllo del flusso per accodare le istruzioni successive.

Se la misurazione su GPU fosse stata effettuata con i classici cronometri della CPU, il sistema avrebbe registrato soltanto il tempo impiegato dalla CPU a inserire il comando nella coda di esecuzione, restituendo latenze apparenti microscopiche, falsate e vicine allo zero.

Per aggirare questo problema, la misurazione su GPU è stata strutturata sfruttando gli eventi CUDA. Nello specifico, il Pre-Hook e l'Hook si occupano di registrare l'allocazione di due eventi hardware direttamente nel flusso di esecuzione della GPU. Successivamente, prima di calcolare la latenza netta tramite la funzione `start_event.elapsed_time(end_event)`, viene invocato un comando di sincronizzazione esplicita. Questo passaggio (`torch.cuda.synchronize()`) costringe la CPU ad attendere la reale conclusione fisica di tutte le operazioni asincrone della GPU. Soltanto dopo questa sincronizzazione forzata i registri temporali hardware vengono letti in modo sicuro, garantendo la validità dei tempi raccolti.

I processori e gli acceleratori hardware moderni presentano spesso anomalie temporali durante i primissimi passaggi di un algoritmo. Questo fenomeno, noto come "overhead da cold start", è dovuto al tempo iniziale richiesto per allocare dinamicamente i tensori nei registri di memoria, caricare i driver grafici e inizializzare i meccanismi di caching interni dei framework di Deep Learning.

Per neutralizzare questo impatto e ottenere misurazioni stabili e affidabili, è stata prevista una fase iniziale di *Warm-up*, durante la quale sono state processate 10 immagini così da rendere le fluttuazioni iniziali isolate ed escluse dalla raccolta dati. Soltanto al termine di questa fase è stata avviata l'inferenza reale sulle restanti immagini di test. I dati raccolti dagli hook sono stati infine strutturati attraverso la libreria `pandas`, creando dei file CSV pronti per essere trasformati nei grafici comparativi della sezione successiva.

4 Risultati sperimentali

In questa sezione vengono presentati e analizzati i dati empirici raccolti durante le sessioni di profiling del modello self-supervised `dinov2_vits14`. L'analisi quantitativa si concentra sul confronto delle performance computazionali dell'architettura transformer durante l'inferenza su CPU e GPU.

4.1 Throughput e latenza

L'esecuzione del benchmark globale evidenzia un divario prestazionale netto tra le due architetture computazionali. L'inferenza eseguita nell'ambiente CPU ha registrato un'efficienza limitata, con un *Throughput* complessivo ridotto a soli **3,54 FPS** (frame per secondo).

Al contrario, la commutazione del runtime su GPU ha sbloccato la natura intrinsecamente parallela del vision transformer. Il Throughput è incrementato notevolmente, raggiungendo il valore di **147,42 FPS**. Simmetricamente, la latenza media per il ciclo d'inferenza di un singolo frame si è ridotta ad appena **6,8 ms** su GPU. Questo ordine di grandezza è compatibile con i requisiti stringenti delle applicazioni Edge AI e Smart Vision operanti in tempo reale, per le quali una latenza ridotta e deterministica costituisce un vincolo critico.

4.2 Analisi dei sottomoduli

Tramite l'utilizzo della tecnica dei *PyTorch Forward Hooks*, è stato possibile scendere a livello di granularità dei layer, estraendo i tempi di esecuzione esatti per i singoli sottomoduli di **Attention** e **MLP** all'interno di ciascuno dei 12 blocchi del transformer.

Analizzando i tempi di esecuzione estratti per la CPU, emerge chiaramente come l'elaborazione del tensore d'ingresso sia ostacolata da severi colli di bottiglia computazionali. La CPU, operando in maniera prevalentemente sequenziale, soffre il carico massivo di calcolo tensoriale e le molteplici moltiplicazioni matriciali (QK^T) insite nella logica di *Self-Attention* e nelle reti dense. I risultati evidenziano una latenza media di circa **9,63 ms** per i blocchi **Attention** e **13,89 ms** per i moduli **MLP**. Sommati lungo l'intera profondità dei 12 blocchi architetturali, questi ritardi inducono una saturazione temporale dell'inferenza, rendendo l'utilizzo di elaboratori CPU meno praticabile per contesti IoT che richiedono reattività.

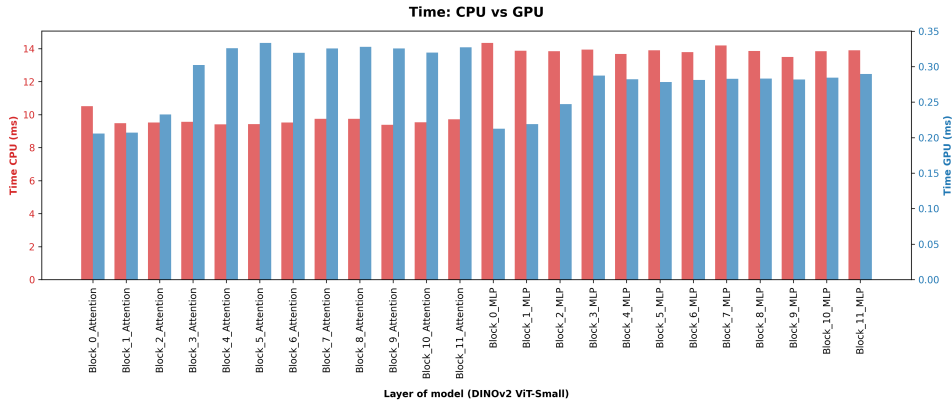


Figura 1: Confronto latenza tra CPU e GPU nei sottomoduli del transformer

Il runtime GPU permette di sfruttare i core paralleli ed in grado di accogliere l'intero tensore dei token spaziali (257×384) e parallelizzare il *Matrix Multiplication* in un numero ridotto di cicli di clock.

I dati sperimentali mostrano che i tempi operativi dei singoli sottomoduli crollano a frazioni di millisecondo: **0,3 ms** di latenza media sia per i layer di **Attention** che per i **MLP**. Questo incremento di efficienza si traduce in uno *speedup* pari a circa **32×** per l'**Attention** e **51×** per le componenti **MLP**.

Inoltre, il grafico di profiling evidenzia un interessante fenomeno di convergenza su GPU in quanto i moduli **Attention** e **MLP** mostrano tempi di esecuzione simili per tutti i 12 blocchi. Questa stabilità conferma che la GPU lavora azzerando l'overhead sequenziale tipico delle CPU e dimostra l'assoluta necessità di acceleratori per il deployment efficace di architetture JEPA-like in ambito Edge.

Tabella 1: Confronto indicativo per sottomodulo (CPU vs GPU)

Sottomodulo	Latenza Media CPU (ms)	Latenza Media GPU (ms)	Speedup Hardware
Block.X.Attention	9,63	0,296	$\approx 32\times$
Block.X.MLP	13,89	0,269	$\approx 51\times$

5 Deployment embedded ed Edge AI

L'analisi empirica condotta sul modello `dinov2_vits14` fornisce evidenze quantitative fondamentali per la progettazione di nodi IoT avanzati. Le metriche prestazionali raccolte definiscono i vincoli operativi per l'integrazione di capacità visive autonome nei sistemi operanti a ridosso della sorgente dati.

I dati estratti dal profiling dimostrano che l'esecuzione di modelli basati su architetture vision transformer su CPU risulta meno compatibile con i requisiti dei sistemi embedded industriali. Nello specifico, il processore impiega 282,3 ms per la computazione di un singolo fotogramma limitando il throughput complessivo a 3,54 FPS. Questo valore è inferiore agli standard minimi richiesti per applicazioni di monitoraggio real time (tipicamente tra i 10 e i 30 fps).

5.1 Criticità del cloud

Storicamente, la limitata capacità di calcolo dei nodi periferici è stata compensata delegando l'elaborazione dei flussi video a server remoti. Tuttavia, questo approccio cloud-centric non è del tutto ottimale. In primo luogo, il trasferimento di payload informativi pesanti è inevitabilmente soggetto a latenze di rete non deterministiche, rendendo il tempo di reazione dell'intero sistema incerto. In secondo luogo, la trasmissione continua di flussi video ad alta definizione satura rapidamente la larghezza di banda disponibile, imponendo costi di infrastruttura elevati in contesti con reti di sensori ad alta densità. Infine, l'esternalizzazione di dati visivi grezzi al di fuori del perimetro locale genera implicazioni per la privacy e l'integrità dei dati, esponendo le informazioni a rischi di intercettazione durante il transito verso l'infrastruttura remota.

5.2 Edge AI

Per superare queste limitazioni strutturali, l'ingegneria dei sistemi smart si sta orientando verso il paradigma dell'Edge AI, elaborando le informazioni visive localmente senza gravare sull'infrastruttura di rete. I risultati sperimentali confermano che la migrazione del carico computazionale permette di abbattere la latenza.

In ambito embedded, il parallelismo viene replicato adottando micro-acceleratori hardware dedicati a bassissimo consumo. Chip specializzati come le Neural Processing Unit (NPU) o le Tensor Processing Unit (TPU), progettati per eseguire matricialmente operazioni in un budget energetico basso, replicano l'efficienza registrata su GPU. Queste architetture abilitano l'inferenza di modelli JEPA all'avanguardia garantendo prestazioni real-time, totale autonomia decisionale in assenza di connettività e tutela della privacy del dato.

6 Conclusioni

L'analisi presentata ha riportato il profilo computazionale del modello `dinov2_vits14` per valutarne la concreta applicabilità nell'ambito dell'Edge computing e dell'IoT. Attraverso l'implementazione metodologica dei forward hooks in PyTorch, è stato possibile isolare e quantificare empiricamente l'onere imposto dalle operazioni tensoriali sui singoli sottomoduli dell'architettura.

L'analisi ha confermato che l'infrastruttura intrinsecamente sequenziale delle CPU tradizionali costituisce un collo di bottiglia sistematico per i calcoli di Self-Attention e per le proiezioni dense dei layer Multi-Layer Perceptron. Al contrario, la parallelizzazione massiva dell'hardware grafico ha dimostrato la capacità di ridurre tale complessità algoritmica, livellando i costi d'esecuzione e aumentando le prestazioni complessive del sistema.

Si evince pertanto che la transizione tecnologica verso Smart Systems realmente autonomi non dipende esclusivamente dall'affinamento del self-supervised learning, ma necessita di una co-progettazione con l'hardware sottostante. Solo attraverso l'adozione strutturale di acceleratori dedicati sarà possibile distribuire intelligenza artificiale visiva di alto livello sui nodi periferici, rispettando i ferrei vincoli di consumo, sicurezza e reattività imposti dal mondo fisico.